Effective Modern C++

CH 01: Deducing Types

▼ **Template type deduction**

```
template<typename T>
void f(ParamType param);

f(expr);  // call f with some expression
```

The type deduced for T is dependent not just on the type of expr, but also on the
form of ParamType. There are three cases:

- **ParamType is a pointer or reference type, but not a universal reference:**
  - If expr's type is a reference or pointer, ignore the reference or pointer part .
  - Then pattern-match expr's type against ParamType to determine T.
- **ParamType is a universal reference:**
  - If expr is an lvalue, both T and ParamType are deduced to be lvalue references.
    That's doubly unusual. First, it's the only situation in template type

deduction where T is deduced to be a reference. Second, although ParamType is declared
using the syntax for an rvalue reference, its deduced type is an lvalue reference.

- If expr is an rvalue, the “normal” (i.e., Case 1) rules apply.

- **ParamType is neither a pointer nor a reference:**
  - As before, if expr’s type is a reference, ignore the reference part.
  - If, after ignoring expr’s reference-ness, expr is const, ignore that, too. If it’s
    volatile, also ignore that.

Array and Function Arguments

During template type deduction, arguments that are array or function names decay to pointers, unless they’re used to initialize references.

```
void someFunc(int, double); // someFunc is a function;
                                                        //
type is void(int, double)

template<typename T>
void f1(T param);          // in f1, param passed by value
template<typename T>
void f2(T& param);         // in f2, param passed by ref


f1(someFunc);             // param deduced as ptr-to-func;
                                                    // typ
e is void (*)(int, double)


f2(someFunc);             // param deduced as ref-to-func;
                                                    // typ
e is void (&)(int, double)
```

▼ Understand auto type deduction

When a variable is declared using auto, auto plays the role of T in the template, and the type specifier for the variable acts as ParamType.

Array and function names undergo pointer decay for non-reference type specifiers, a behavior that also occurs during `auto` type deduction.

```
const char name[] = "R. N. Briggs";     // name's type is
const char[13]
auto arr1 = name;                      // arr1's type is c
onst char*
auto& arr2 = name;                    // arr2's type is co
nst char (&)[13]
```

The treatment of braced initializers is the only way in which auto type deduction and
template type deduction differ. When an auto–declared variable is initialized with a
braced initializer, the deduced type is an instantiation of std::initializer_list. But if the corresponding template is passed the same initializer, type deduction fails,
and the code is rejected. So the only real difference between auto and template type deduction is that auto assumes that a braced initializer represents a std::initializer_list, but template type deduction doesn't.

auto in a function return type or a lambda parameter implies template type deduction, not auto type deduction.

▼ Understand decltype

decltype typically parrots back the exact type of the name or expression.

```
bool f(const Widget& w);  // decltype(w) is const Widget&
                                                    // dec
ltype(f) is bool(const Widget&)
```

In C++11, perhaps the primary use for decltype is declaring function templates where the function's return type depends on its parameter types.

```
// In C++ 11
template<typename Container, typename Index> // works, but
requires refinement
auto authAndAccess(Container& c, Index i) -> decltype(c
[i])
{
    authenticateUser();
    return c[i];
}

/*
C++11 permits return types for single-statement lambdas to
be deduced, and C++14
extends this to both all lambdas and all functions, includ
ing those with multiple
statements. In the case of authAndAccess, that means that
in C++14 we can omit the
trailing return type, leaving just the leading auto.
*/

// In C++ 14 (wrong)
template<typename Container, typename Index>
auto authAndAccess(Container& c, Index i)
{
    authenticateUser();
    return c[i]; // return type deduced from c[i]
}

/*
Here, d[5] returns an int&, but auto return type deduction
for authAndAccess will
strip off the reference, thus yielding a return type of in
t. That int, being the return
value of a function, is an rvalue, and the code above thus
attempts to assign 10 to an
```

```
rvalue int. That’s forbidden in C++, so the code won’t com
pile.
*/

/*
To get authAndAccess to work as we’d like, we need to use
decltype type deduction
for its return type, i.e., to specify that authAndAccess s
hould return exactly the same
type that the expression c[i] returns. The guardians of C+
+, anticipating the need to
use decltype type deduction rules in some cases where type
s are inferred, make this
possible in C++14 through the decltype(auto) specifier.
*/


// In C++ 14 (correct)
template<typename Container, typename Index>
decltype(auto) authAndAccess(Container& c, Index i)
{
    authenticateUser();
    return c[i];
}
```

The use of decltype(auto) is not limited to function return types. It can also be convenient for declaring variables when you want to apply the decltype type deduction rules to the initializing expression:

```
Widget w;
const Widget& cw = w;
auto myWidget1 = cw; // auto type deduction: myWidget1's t
ype is Widget
decltype(auto) myWidget2 = cw; // decltype type deduction:
const Widget&
```

Wrapping the name x in parentheses—“(x)”—yields an expression more complicated than a name. Being a name, x is an lvalue, and C++ defines the expression (x) to be an lvalue, too.
decltype((x)) is therefore int&. Putting parentheses around a name can change the type that decltype reports for it!

```
decltype(auto) f1()
{
    int x = 0;
    …
    return x; // decltype(x) is int, so f1 returns int
}


decltype(auto) f2()
{
    int x = 0;
    …
    return (x); // decltype((x)) is int&, so f2 returns in
t&
}
```

▼ Know how to view deduced types

```
#include <boost/type_index.hpp>
template<typename T>
void f(const T& param)
{
using std::cout;
using boost::typeindex::type_id_with_cvr;
// show T
cout << "T = "
<< type_id_with_cvr<T>().pretty_name()
<< '\n';
// show param's type
```

```
cout << "param = "
<< type_id_with_cvr<decltype(param)>().pretty_name()
<< '\n';
…
}
```

The way this works is that the function template boost::typeindex::type_id_with_cvr takes a type argument (the type about which we want informa-tion) and doesn't remove const, volatile, or reference qualifiers (hence the "with_cvr" in the template name).

The result is a boost::typeindex::type_index object, whose pretty_name member function produces a std::string containing a human-friendly representation of the type.

CH 02: auto

▼ **Prefer auto to explicit type declarations**

auto / std::function object can represent types known only to compilers as shown below:

```
// comparison func for Widgets pointed to by std::unique_p
trs using auto
auto derefUPLess = [](const std::unique_ptr<Widget>& p1, c
onst std::unique_ptr<Widget>& p2) { return *p1 < *p2; };


/*
std::function is a template in the C++11 Standard Library
that generalizes the idea
of a function pointer. Whereas function pointers can point
only to functions, how-
ever, std::function objects can refer to any callable obje
ct, i.e., to anything that can
be invoked like a function. Just as you must specify the t
```

```
ype of function to point to
when you create a function pointer (i.e., the signature of
the functions you want to
point to), you must specify the type of function to refer
to when you create a
std::function object. You do that through std::function's
template parameter.
*/

// comparison func for Widgets pointed to by std::unique_p
trs using function object
std::function<bool(const std::unique_ptr<Widget>&,
const std::unique_ptr<Widget>&)>
derefUPLess = [](const std::unique_ptr<Widget>& p1, const
std::unique_ptr<Widget>& p2) { return *p1 < *p2; };
```

An auto-declared variable holding a closure has the same type as the closure, and as such
it uses only as much memory as the closure requires. The type of a std::function declared variable holding a closure is an instantiation of the std::function template, and that has a fixed size for any given signature. This size may not be adequate for the closure it's asked to store, and when that's the case, the std::function constructor will allocate heap memory to store the closure.

The result is that the std::function object typically uses more memory than the auto-declared object. And, thanks to implementation details that restrict inlining and yield indirect function calls, invoking a closure via a std::function object is almost certain to be
slower than calling it via an auto-declared object.

`std::vector<int>::size_type` is an unsigned type whose size depends on the platform, but `unsigned` is always 32-bit on Windows. So code using `unsigned` instead of `size_type` may work on 32-bit Windows but break on 64-bit Windows due to size mismatches. Using the proper `size_type` avoids these portability bugs.

```
auto sz = v.size(); // sz's type is std::vector<int>::size
_type
```